

PROJET STM32: RADAR DE VITESSE



Noms : ALBERTS, SINACER

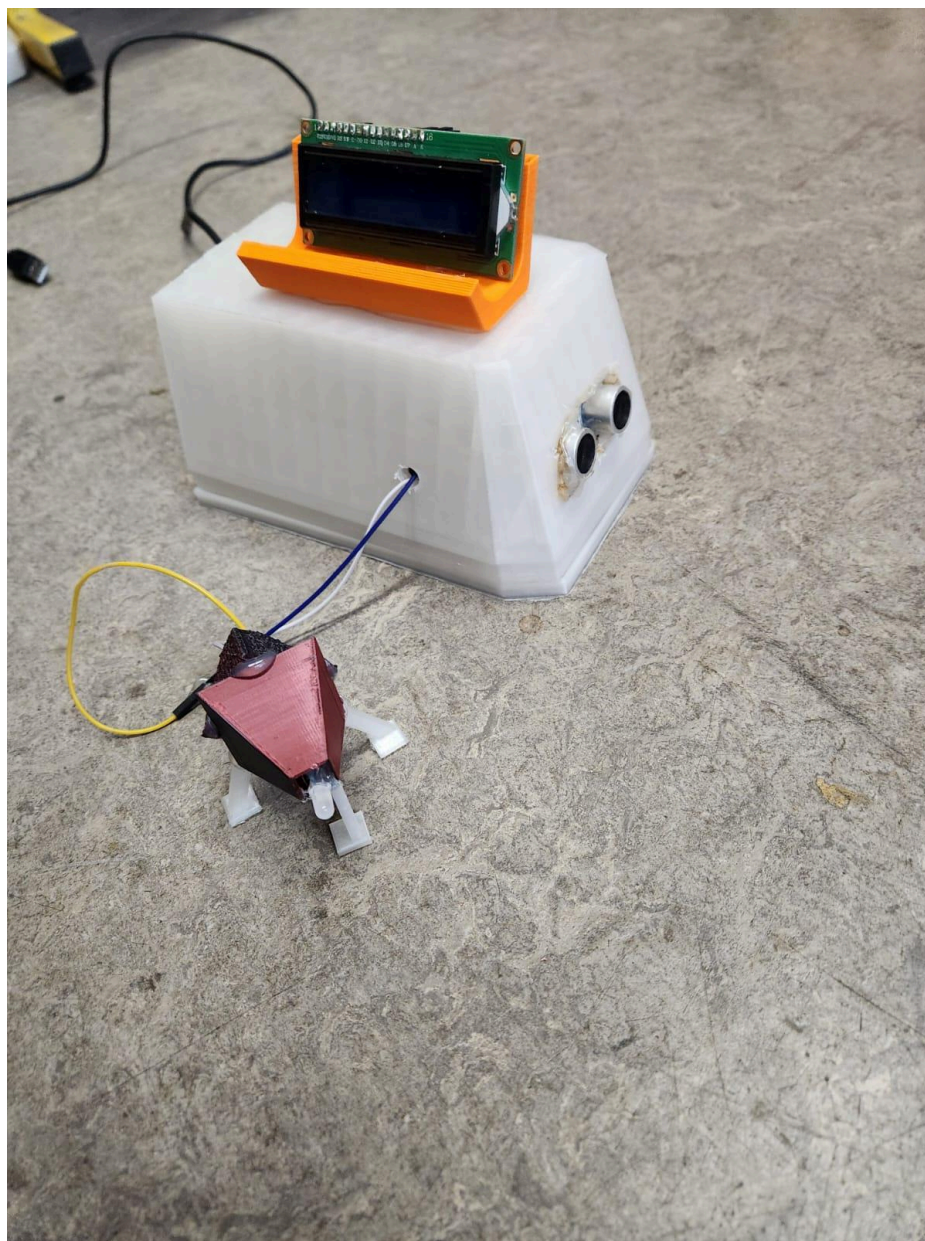
Prénoms : Greg, Tedj El Moulk.

Ing2 instrumentation

Introduction :

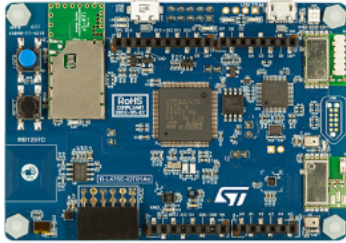
Dans le cadre du projet Microcontrôleur 2, nous avons réalisé un radar de vitesse routier en utilisant la carte STM32L75F. Ce radar repose sur un capteur ultrason HC-SR04 pour détecter un objet en mouvement et calculer sa vitesse. Un afficheur LCD 16 bits est utilisé pour afficher la vitesse de l'objet détecté ainsi qu'un message en cas d'excès de vitesse. De plus, une LED clignote lorsqu'un dépassement de vitesse est détecté.

Voici une photo regroupant tous les composants présents dans ce projet avec leur descriptif ensuite :



Matériels

- Carte STM32L75F :



- Capteur ultrason HC-SR04 :



- Lcd 16 bits :



- Led blanche :



Explications du projet

Nous avons programmé la carte STM32L75F pour contrôler le capteur ultrason HC-SR04, qui détecte un objet en mouvement et mesure la distance parcourue entre deux instants. En utilisant ces mesures, la vitesse de l'objet est calculée en fonction du temps écoulé entre les deux positions. Cette vitesse est ensuite affichée sur l'écran LCD. Si elle dépasse 7 km/h, un message d'excès de vitesse s'affiche sur l'écran, et la LED clignote pour signaler l'infraction. Comme on peut le voir sur la vidéo en annexe à la fin de ce document (led blanche qui flash très rapidement lors de l'excès de vitesse sur la première tentative).

Nous avons une première fonction permettant le calcul d'un délai en microsecondes afin que le signal trig envoyé par le capteur ultrason ait le temps de revenir sur le capteur ultrason. Nous l'avons faite avec un timer s'incrémentant à la microseconde avec un préscalier de 79 et une fréquence d'échantillonnage de 80 MHz :

```
441 /* USER CODE BEGIN 4 */
442 void delay(uint32_t time)
443 {
444     __HAL_TIM_SET_COUNTER(&htim1, 0);
445     while(__HAL_TIM_GET_COUNTER(&htim1) < time);
446 }
```

Ensuite, une seconde fonction nous permet de lire le temps que le signal trig a pris pour revenir jusqu'au capteur ultrason et ainsi nous serons en capacité de déterminer la distance à laquelle se trouve l'objet dans le champ du capteur :

```
448 uint32_t hcsr04_read()
449 {
450     local_time=0;
451     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3, GPIO_PIN_RESET); // reset the TRIG pin
452     delay(2); // wait for 2 µs
453
454     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3, GPIO_PIN_SET); // pull the TRIG pin HIGH
455     delay(10); // wait for 10 µs
456     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3, GPIO_PIN_RESET); // pull the TRIG pin low
457
458     // read the time for which the pin is high
459
460     while (!(HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_4))); // wait for the ECHO pin to go high
461     while (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_4)) // while the pin is high
462     {
463         local_time++; // measure time for which the pin is high
464         delay(1); // wait for 1 µs
465     }
466
467     return local_time;
468 }
```

Nous utilisons cette fonction afin de calculer la distance en multipliant le temps de parcours du signal par la vitesse d'une onde dans l'air (340 m/s).

Nous nous servons de ce calcul dans la troisième fonction qui permet de calculer la vitesse d'un objet (véhicule) en trouvant les 2 distances au radar de cet objet et en calculant la différence divisé par un temps DELTA_T défini en annexe dans les #defines :

```

470 void calcul_speed()
471 {
472     sensor_time = hcsr04_read();
473     distance_1 = (float) ((float) sensor_time * 0.034);
474
475     __HAL_TIM_SET_COUNTER(&htim2, 0);
476
477     while(__HAL_TIM_GET_COUNTER(&htim2) < (DELTA_T * 1000)) { };
478
479     sensor_time = hcsr04_read();
480     distance_2 = (float) ((float) sensor_time * 0.034);
481
482     if((distance_2 > distance_1)
483         && (distance_1 > 20) && (distance_1 < 150)
484         && (distance_2 > 20) && (distance_2 < 150)) // for the speed to be positive and correct
485     {
486         speed = ((distance_2 - distance_1) * 10 / DELTA_T) * 3.6; // Speed in km/h
487
488         if(speed < 1.0)
489         {
490             speed = 0; // Because the object is not moving
491         }
492     }
493     else
494     {
495         speed = 0;
496     }
497
498     sprintf(speed_string, "%f", speed);
499 }

```

Enfin, une quatrième et dernière fonction, qui nous permet d'effectuer un flash lumineux avec notre led que nous activerons lorsque le véhicule est en excès de vitesse :

```

501 void flash_car()
502 {
503     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
504     HAL_Delay(5); // wait for 5 ms before turning off the flash (LED)
505     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
506 }

```

Puis, il y a le main dans lequel nous utilisons ces différentes fonctions (ainsi que l'initialisation de l'afficheur LCD) afin de créer l'algorithme permettant le bon fonctionnement du radar de vitesse :

```

130  /* USER CODE BEGIN 2 */
131  /* --- Start : Launching setup --- */
132  // Start the timers
133  HAL_TIM_Base_Start(&htim1);
134  HAL_TIM_Base_Start(&htim2);
135
136  // Lcd_PortType ports[] = { D4_GPIO_Port(LCD pin), D5_GPIO_Port(LCD pin), D6_GPIO_Port(LCD pin), D7_GPIO_Port(LCD pin)};
137  Lcd_PortType ports[] = {GPIOA, GPIOA, GPIOA, GPIOA};
138
139  // Lcd_PinType pins[] = {D4_Pin(LCD pin), D5_Pin(LCD pin), D6_Pin(LCD pin), D7_Pin(LCD pin)};
140  Lcd_PinType pins[] = {GPIO_PIN_15, GPIO_PIN_2, GPIO_PIN_7, GPIO_PIN_6};
141
142  Lcd_HandleTypeDef lcd;
143
144  // Lcd_create(ports, pins, RS_GPIO_Port, RS_Pin, E_GPIO_Port, E_Pin, LCD_4_BIT_MODE);
145  lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_0, GPIOA, GPIO_PIN_3, LCD_4_BIT_MODE);
146
147  Lcd_cursor(&lcd, 0,1);
148  Lcd_string(&lcd, "Speed (km/h) : ");
149  Lcd_cursor(&lcd, 1,1);
150  Lcd_string(&lcd, "0");
151
152  HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
153  HAL_Delay(1000);
154  HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
155  Lcd_clear(&lcd);
156  /* --- End : Launching setup --- */
157
158  /* USER CODE END 2 */

160  /* Infinite loop */
161  /* USER CODE BEGIN WHILE */
162  while (1)
163  {
164      /* USER CODE END WHILE */
165
166      /* USER CODE BEGIN 3 */
167      uint32_t currentTime = HAL_GetTick();
168
169      Lcd_cursor(&lcd, 0,1);
170      Lcd_string(&lcd, "Speed : ");
171
172      calcul_speed();
173
174      if(mostSpeedin500uSeconds < speed)
175      {
176          mostSpeedin500uSeconds = speed;
177      }
178
179      /* START : --- PRINTF --- */
180      printf("sensor_time = %ld\tdistance_1 = %f\tdistance_2 = %f\tspeed = %f\n", sensor_time, distance_1, distance_2, speed); // @suppress("Float fo
181      /* END : --- PRINTF --- */
182
183      if((currentTime - lastUpdateTime) >= 500)
184      {
185          lastUpdateTime = currentTime;
186
187          Lcd_cursor(&lcd, 1,1);
188          Lcd_string(&lcd, speed_string);
189          Lcd_cursor(&lcd, 1,12);
190          Lcd_string(&lcd, "km/h");
191
192          mostSpeedin500uSeconds = 0;
193      }

194
195      if((speed > MAX_SPEED) && (speed < ERROR_SPEED))
196      {
197          flash_car();
198
199          Lcd_clear(&lcd);
200          Lcd_cursor(&lcd, 0,1);
201          Lcd_string(&lcd, "Over the ");
202          Lcd_cursor(&lcd, 1,3);
203          Lcd_string(&lcd, "speed limit !");
204
205          HAL_Delay(1000);
206          Lcd_clear(&lcd);
207
208          Lcd_cursor(&lcd, 0,1);
209          Lcd_string(&lcd, "Excessive speed");
210          Lcd_cursor(&lcd, 1,1);
211          Lcd_string(&lcd, speed_string);
212          Lcd_cursor(&lcd, 1,12);
213          Lcd_string(&lcd, "km/h");
214
215          HAL_Delay(2000);
216          Lcd_clear(&lcd);
217
218          mostSpeedin500uSeconds = 0;
219      }
220  }
221  /* USER CODE END 3 */
222  }

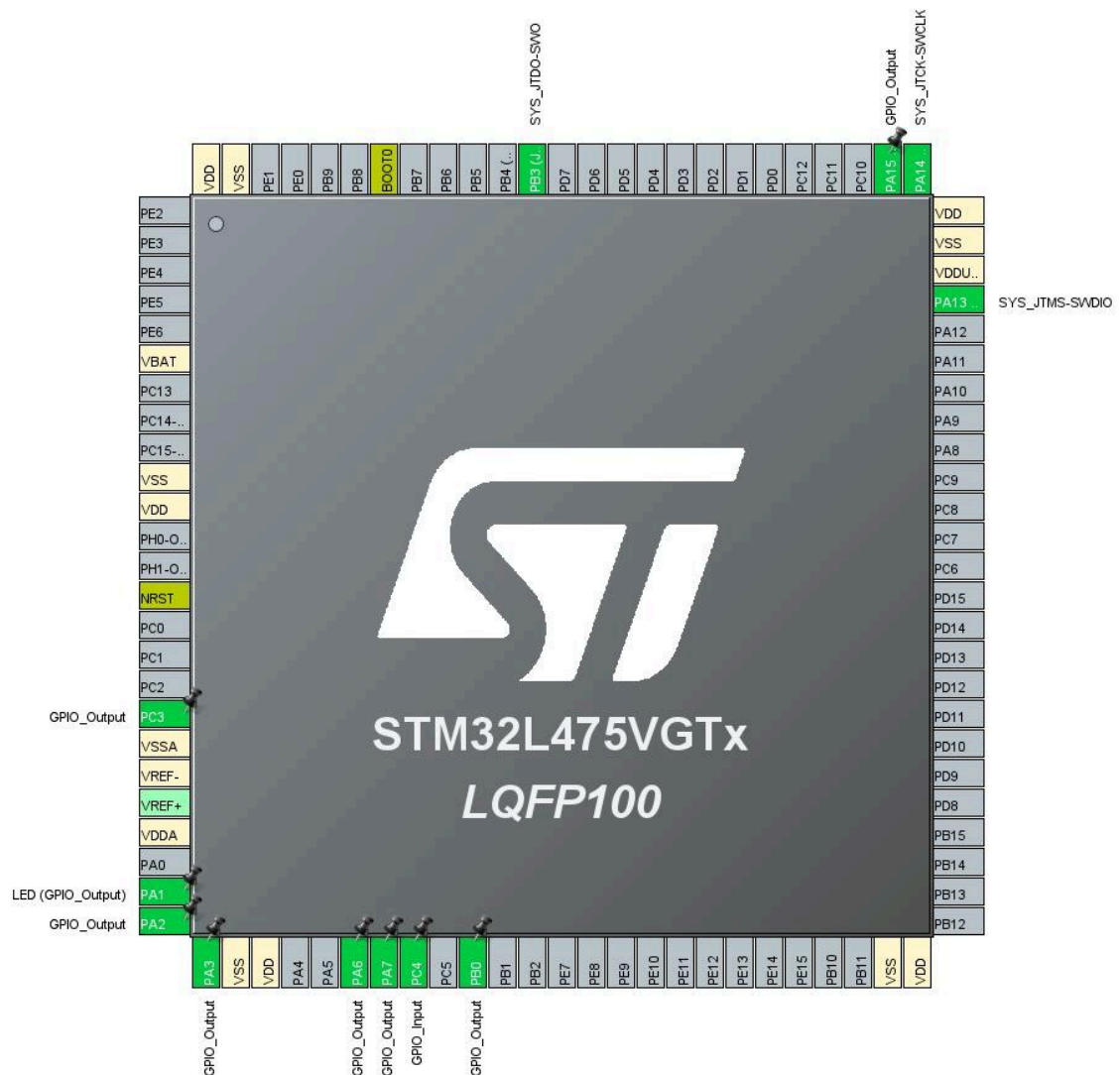
```

Conclusion

Ce projet nous a permis de découvrir et mettre en place un système réel. En effet, nous avons pu rencontrer des difficultés concernant les contraintes de temporalités (concernant les mesures en temps réelles du radar de vitesse). De plus, les contraintes liées à la performance de notre capteur ainsi que ses limitations nous ont permis de comprendre les difficultés d'un tel projet. Nous avons donc pu avoir une idée d'application typique que l'on pourrait avoir en entreprise dans le futur et également avoir un projet à rajouter sur notre CV.

Annexes

Configuration du .ioc :



#Defines des constantes et déclarations des fonctions :

```
37⊖ /* Private define ----- */
38 /* USER CODE BEGIN PD */
39 #define DELTA_T 200 // 200 ms
40 #define MAX_SPEED 7 // 7 km/h MAX !
41 #define ERROR_SPEED 20 // 20 km/h
42
43 /* To wait some micro-seconds */
44 void delay(uint32_t);
45
46 /* To get the time captured */
47 uint32_t hcsr04_read();
48
49 /* To calcul the speed of the object */
50 void calcul_speed();
51
52 /* To flash the overspeeding car */
53 void flash_car();
54
55 /* USER CODE END PD */
```

Vidéo du radar de vitesse routier (en fonctionnement). Flash de la voiture sur le premier passage puis 2 passages en dessous des limites de vitesses (< 7km/h) :

<https://youtu.be/YVXjxsQCurk> (← Lien à cliquer)